

**Yogesh Haribhau Kulkarni** • YouAI Advisor (Helping organizations in their AI journeys) | PhD (Geometric Modeling) | Tech Columnist (...)
now •

...

New article out: Text vs. Graph Transformer, a component-by-component comparison.

When I started working on graph neural networks for geometric data, I kept hitting the same wall: most approaches assume you either work on Transformers OR graphs, never both.

So I wrote the article as notes of my own understanding. It goes through every major component side by side:

- Input embeddings: one vocab lookup vs. three tensors (nodes, edges, adjacency)
- Positional encoding: sinusoidal integers vs. Laplacian eigenvectors and why rotation-invariance makes the graph case genuinely hard
- Self-attention: dense by default in text, a design choice in graphs, sparse, dense, or hybrid
- Cross-attention: same mechanism, richer applications (graph-to-graph transforms, geometry + language)
- Encoder-only / decoder-only / encoder-decoder: all three families exist in graph world, with one extra complication, graphs have no canonical node ordering, which makes generation much harder than it looks

Written for technical freshers who know ML basics and are curious about why Graph Transformers are not just "Transformers on graphs", they require fundamentally different solutions at almost every layer.

The longer I work at the intersection of geometric modeling and ML, the more I believe graph-structured data is the next major frontier after language and vision.

Full article in the comments.

If you are working on Graph Transformers, Geometric Deep Learning, I would genuinely like to hear what problems you are running into. The open questions here are real and interesting.

#GraphTransformers #GeometricDeepLearning #MachineLearning #EDA #ChipDesign #GNN
 #PyTorchGeometric #AlinEngineering #gdl

Component	Text Transformer	Graph Transformer
Input	Token sequence, one matrix	Nodes + edges + adjacency, three tensors
Input embeddings	Lookup table (discrete vocab)	Linear projection (continuous or categorical node/edge features)
Positional encoding	Sinusoidal or learned integer-position PE	LapPE, RWSE, distance encoding, or equivariant coordinate encoding
Self-attention	Dense, all tokens attend to all tokens	Dense (full) or sparse (neighbourhood-only) or hybrid
Edge features	Not applicable	Modify attention scores and value aggregation
Cross-attention	Decoder attends to encoder output	Same, but over node sets; also used in graph-to-graph tasks
Encoder-only	BERT — classification, understanding	Graph property prediction, node classification
Decoder-only	GPT — text generation	Autoregressive graph generation (hard ordering problem)
Encoder-decoder	T5 — translation, summarisation	Graph-to-graph transforms, graph-to-text